



Análise e Desenvolvimento de Sistemas
Modelagem de Banco de Dados e NoSQL
Orientações para o Relatório Final

Nome: Vitória Freitas

RA: 2523279

Polo de matrícula: Carapicuíba - Vila Dirce

Ano da postagem: 2026

Título da aula: Introdução e Fundamentação de Banco de Dados

Roteiro 1

Descrição do estudo de caso desenvolvido:

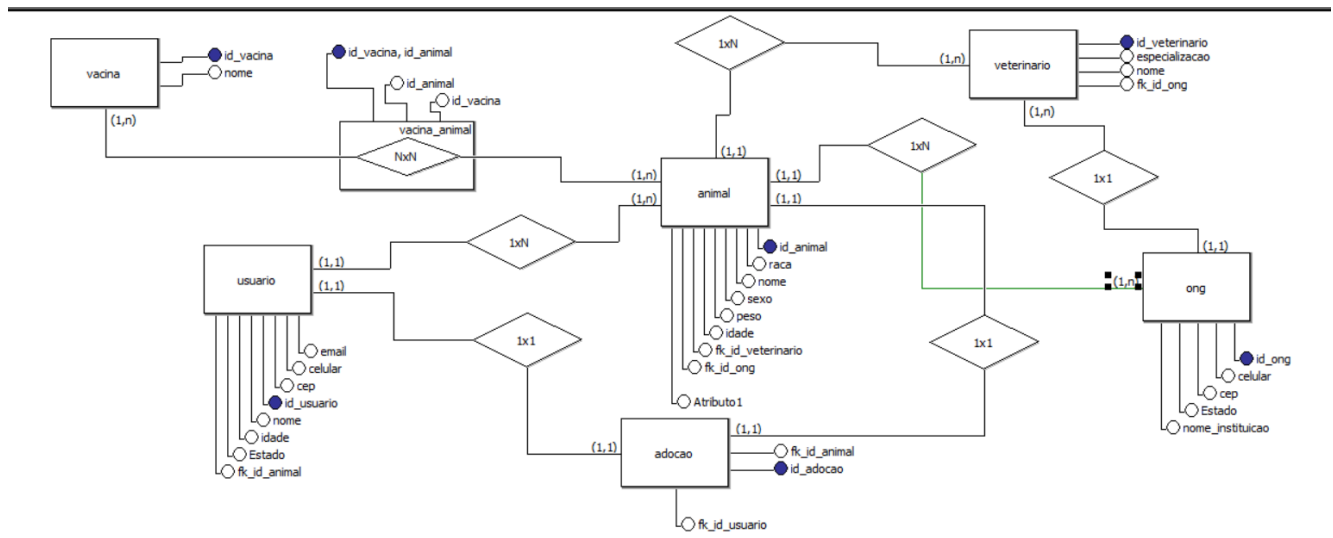
Desenvolvemos um projeto de banco de dados para um aplicativo, que teria o objetivo de gerenciar processos de adoção de animais.

O aplicativo tinha que cumprir as seguintes funções: cadastro de animais, cadastro dos usuários, cadastro de ONGs, gerenciamento de histórico de vacinas, acompanhamento dos veterinários e registro de solicitações das adoções.

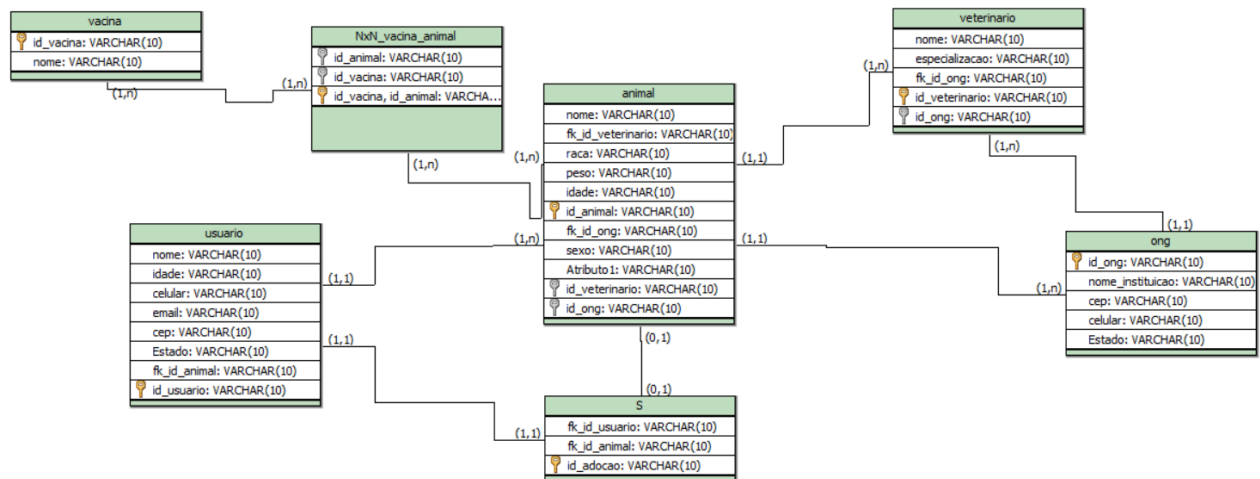
Nomes das principais entidades criadas no modelo conceitual, lógico e físico foram:

- animal
- vacina
- usuario
- adocao
- veterinario
- ong

Diagrama do modelo conceitual (DER):



Representação do modelo lógico:



Scripts da implementação do modelo físico no SQL Server:

--criação de tabelas, as foreign key são adicionadas depois

```

create table animal(
    id_animal int identity primary key,
    raca varchar(20) not null,
    sexo char(1) not null,
    especie varchar(20) not null,
    nome varchar(30) not null,
    peso numeric(5, 2) not null,
    idade int not null,
    fk_id_veterinario int,
    fk_id_ong int
)
go
  
```

```

create table vacina(
    id_vacina int identity primary key,
    nome varchar(50) not null,
)
go
  
```

-- tabela intermediaria

```

create table vacina_animal(
    id_vacina int,
    id_animal int,
    primary key(id_vacina, id_animal)
)
go
  
```

```

create table ong(
    id_ong int identity primary key,
    celular varchar(30) not null,
    cep varchar(30) not null,
    Estado char(2) not null,
    nome_instituicao varchar(30) not null,
  
```

```

)
go

create table veterinario(
    id_veterinario int identity primary key,
    especializacao varchar(30) not null,
    cep varchar(30) not null,
    nome varchar(30) not null,
    fk_id_ong int
)
go

create table usuario(
    id_usuario int identity primary key,
    email varchar(30) not null,
    celular varchar(30) not null,
    cep varchar(10) not null,
    nome varchar(30) not null,
    idade int,
    Estado char(2) not null,
    fk_id_animal int
)
go

create table adocao(
    id_adocao int identity primary key,
    fk_id_animal int,
    fk_id_usuario int
)
go

--foreign keys

--animal
alter table animal add constraint fk_animal_veterinario
foreign key(fk_id_veterinario) references veterinario(id_veterinario)
go

alter table animal add constraint fk_animal_ong
foreign key(fk_id_ong) references ong(id_ong)
go

--vacina_animal
alter table vacina_animal add constraint fk_vacina_animal
foreign key(id_vacina) references vacina(id_vacina)
go

alter table vacina_animal add constraint fk_animal_vacina
foreign key(id_animal) references animal(id_animal)
go

--veterinario
alter table veterinario add constraint fk_vet_ong

```

```

foreign key(fk_id_ong) references ong(id_ong)
go

--usuario

alter table usuario add constraint fk_usuario_animal
foreign key(fk_id_animal) references animal(id_animal)
go

--adocao

alter table adocao add constraint fk_adocao_animal
foreign key(fk_id_animal) references animal(id_animal)
go

alter table adocao add constraint fk_adocao_usuario
foreign key(fk_id_usuario) references usuario(id_usuario)
go

```

Imagem de dados de algumas tabelas através de joins:

	nome_instituicao	nome	especie	raca
1	abrigo cachorros e gatos	Jolly	Canino	Golden
2	abrigo cachorros e gatos	Max	Canino	SRD
3	vida animal	Nala	Felino	SRD
4	aves e repteis reservatorio	Alan	Aves	Calopsita
5	instituição casa	Leo	Repteis	Gecko Leopardo

	nome	vacina
1	Jolly	vacina x
2	Max	vacina x
3	Nala	vacina x
4	Alan	vacina y
5	Leo	vacina y

	nome	nome
1	Jolly	Henrique Ferreira
2	Max	Julia Santos
3	Alan	Gabriel Antonio
4	Leo	Ana Paula
5	Nala	Luciano Lopes

Script dos joins mostrados acima:

```

select o.nome_instituicao, a.nome, a.especie, a.raca
from ong o

```

```

inner join animal a
    on a.fk_id_ong = o.id_ong
go

select a.nome, v.nome as 'vacina'
from animal a
inner join vacina_animal va
    on va.id_animal = a.id_animal
inner join vacina v
    on v.id_vacina = va.id_vacina
go

select a.nome, u.nome
from animal a
inner join adocao ad
    on ad.fk_id_animal = a.id_animal
inner join usuario u
    on ad.fk_id_usuario = u.id_usuario
go

```

Evolução dos bancos de dados e sua importâncias nas empresas:

O início da história dos bancos de dados começa na década de 40. Nessa época os dados eram guardados em papéis. Fichas eram preenchidas para guardar os dados, essas fichas eram guardadas em pastas e as pastas eram guardadas em armários de metal.

Na década de 50 esse processo foi digitalizado, graças à evolução dos computadores. Então na década de 60 em uma conferência, foi definido um molde para os bancos de dados, que são compostos por 4 partes: base de dados, sistema gerenciador, linguagem de exploração e programas adicionais. Foram desenvolvidos dois principais modelos de dados: modelo de rede (CODASYL) e o modelo hierárquico (IMS).

O modelo relacional foi proposto por Edgar F.Codd, um pesquisador da IBM na década de 70. Principais protótipos de sistema relacional na mesma década foram: Ingres (desenvolvido pela UCB) e System R (Desenvolvido pela IBM). Também foi criado o modelo ER (Entidade-Relacionamento) pelo Dr.Peter Chan.

Em suma, a importância dos bancos de dados em uma empresa se dá pela significância da organização e manutenção dos dados em uma instituição.

Referências:

<https://medium.com/@pedroivoflorentino/a-origem-e-evolu%C3%A7%C3%A3o-dos-bancos-de-dados-72be6480c94>

<https://www.devmedia.com.br/a-historia-dos-banco-de-dados/1678>

Título da aula: Construção do Modelo Físico e Comandos DDL

Roteiro 2

Scripts SQL utilizados na criação das tabelas com chaves primárias:

```
create table Editoras(  
    id_editora int primary key identity,  
    nome varchar(30),  
    email varchar(30),  
    celular varchar(30)  
)  
go  
  
create table Livros(  
    id_livro int primary key identity,  
    nome varchar(30),  
    ano_lancamento datetime,  
    fk_editora int  
)  
go  
  
create table Autores(  
    id_autor int primary key identity,  
    nome varchar(30),  
    idade int,  
    email varchar(30)  
)  
go  
  
create table Livro_Autor(  
    fk_id_autor int,  
    fk_id_livro int,  
    primary key(fk_id_autor, fk_id_livro)  
)  
go
```

```
create table Detalhes_Livro(  
    id_detalhes_livros int primary key identity,  
    detalhes varchar(100),  
    fk_livro int unique  
)  
go
```

Scripts de criação de chaves estrangeiras:

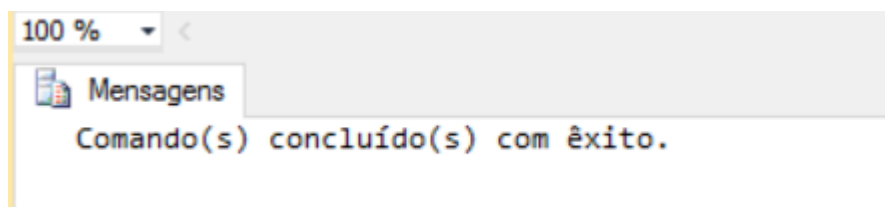
```
alter table Livros add constraint fk_livros_editora  
foreign key(fk_editora) references Editoras(id_editora)  
go
```

```
alter table Livro_Autor add constraint livro_autor_livro  
foreign key(fk_id_livro) references Livros(id_livro)  
go
```

```
alter table Livro_Autor add constraint livro_autor_autor  
foreign key(fk_id_autor) references Autores(id_autor)  
go
```

```
alter table Detalhes_Livro add constraint Detalhes_Livro_Livros  
foreign key(fk_livro) references Livros(id_livro)  
go
```

Capturas de tela da execução dos comandos no SQL Server:



100 % <

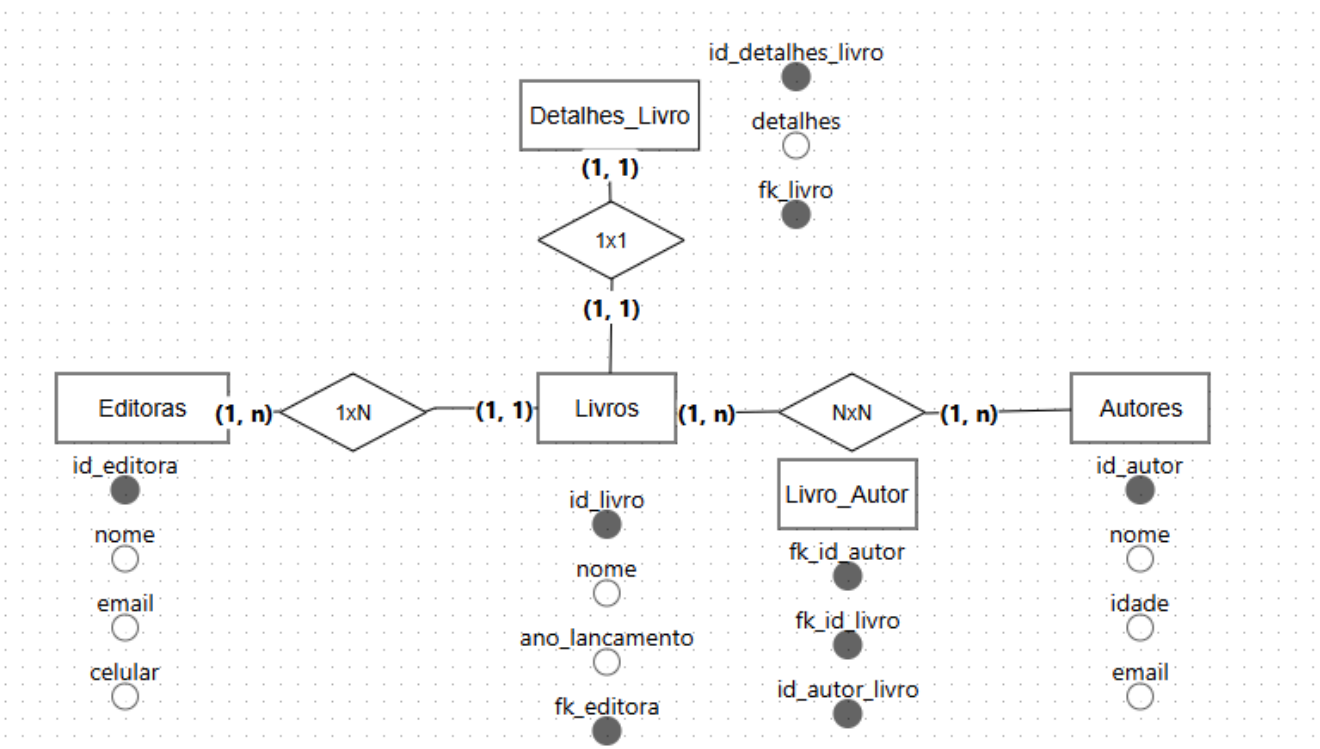
Resultados Mensagens

	nome	nome	ano_lancamento
1	Livro X	Antonio Fernando	2026-06-12 00:00:00.000
2	Livro Y	Marianna Karla	2021-01-11 00:00:00.000
3	Livro Z	Julio Leandro	2016-06-05 00:00:00.000
4	Livro I	Maria Ferraz	2012-07-07 00:00:00.000

	nome	nome
1	Editora X	Livro X
2	Editora Y	Livro Y
3	Editora Z	Livro Z
4	Editora I	Livro I

	nome	detalhes
1	Livro X	Livro sobre fantasia
2	Livro Y	Romance contemporâneo
3	Livro Z	Ficção científica
4	Livro I	Fantasia medieval

Representação do relacionamento entre as tabelas criadas:



Tipos de cardinalidade implementados:

- Cardinalidade 1xN: Uma instância pode se relacionar com outras instâncias, mas a outra só pode relacionar com uma. Isso ocorre na tabela Editoras com a tabela Livros, uma editora pode possuir diversos livros, mas um livro apenas pode possuir uma editora.

- Cardinalidade 1x1: Uma instância se relaciona apenas com uma instância, isso ocorre na tabela Livros com Detalhes_Livro, onde um livro apenas pode possuir um detalhe e um detalhe só pode possuir um livro.
- Cardinalidade NxN: Uma instância pode se relacionar com diversas outras instâncias e a outra pode fazer o mesmo. Podemos ver esse tipo de relacionamento ocorrer na tabela Livros com a Autores, um livro pode possuir diversos autores e os autores podem ter diversos livros. Quando esse tipo de relacionamento ocorre, precisamos ter uma tabela associativa, como a Livro_Autor que vai fazer a ligação entre Livros e Autores.

Referências:

<https://www.dio.me/articles/tem-medo-da-cardinalidade-em-banco-de-dados-agora-nao-mais-se-liga-nesse-artigo>

Título da aula: Manipulação de Dados com Comandos DML (Parte I)

Roteiro 3

Scripts SQL utilizados durante a atividade

```
--Tabelas do banco
create table Clientes(
    id_cliente int primary key identity,
    nome varchar(30),
    telefone varchar(30),
    cep varchar(20),
    casa_numero int
)
go
create table Pizzas(
    id_pizza int primary key identity,
    nome varchar(30),
    preco numeric(5, 2)
)
go
create table Pedidos(
    fk_id_cliente int,
    fk_id_pizza int,
    status varchar(30),
    primary key(fk_id_cliente, fk_id_pizza)
)
go

alter table Pedidos add constraint cliente_pedido
foreign key(fk_id_cliente) references Clientes(id_cliente)
go
alter table Pedidos add constraint pizza_pedido
foreign key(fk_id_pizza) references Pizzas(id_pizza)
go
```

--Comandos insert, update e delete

--Inserindo pizzas no banco

```
insert into Pizzas values('Margherita', 53.2)
insert into Pizzas values('Calabresa', 45.5)
insert into Pizzas values('Portuguesa', 48.7)
insert into Pizzas values('Pizza da Casa', 50.6)
go
```

--Inserindo clientes no banco, dados: nome, telefone, cep e número da casa

```
insert into Clientes values('João', '975662243', '6453-452', 45)
insert into Clientes values('Maria', '989804753', '4323-744', 7)
go
```

--Inserindo pedidos, dados: id do cliente, id da pizza desejada e status

```
insert into Pedidos values(1, 2, 'Fazendo')
insert into Pedidos values(2, 3, 'Fazendo')
insert into Pedidos values(1, 1, 'Cancelado')
go
```

--Atualizando preço das pizzas

```
update Pizzas set preco = (preco * 0.1) + preco
go
```

```
update Clientes set cep = '76314-643', casa_numero = 42 where id_cliente = 1
go
```

--Deletando pedidos cancelados

```
delete Pedidos where status = 'Cancelado'
go
```

--Desafio do roteiro

```
update Pizzas set nome = 'Superpromoção' where nome = 'Pizza da Casa'
go
update Pizzas set preco = preco - (preco * 0.15) where nome = 'Superpromoção'
go
```

Comando mostrando um join com a tabela de Pizzas, Cliente e Pedidos:

	Pizza	Preço	Cliente
1	Margherita	55.00	João
2	Calabresa	49.50	João
3	Portuguesa	38.50	Maria

Capturas de tela demonstrando a execução dos comandos

(1 linha(s) afetadas)

(1 linha(s) afetadas)

(1 linha(s) afetadas)

(1 linha(s) afetadas)

(1 linha(s) afetadas)

(1 linha(s) afetadas)

Evidências das alterações realizadas nas tabelas

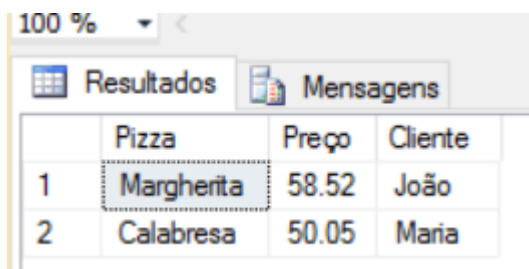
Tabela Pizzas antes do update:

	id_pizza	nome	preco
1	8	Margherita	53.20
2	9	Calabresa	45.50
3	10	Portuguesa	48.70
4	11	Pizza da Casa	50.60

Tabela Pizzas depois do update:

	id_pizza	nome	preco
1	8	Margherita	58.52
2	9	Calabresa	50.05
3	10	Portuguesa	53.57
4	11	Superpromoção	47.31

Comando join com a tabela de Pizzas, Cliente e Pedidos, depois do delete e update:



	Pizza	Preço	Cliente
1	Margherita	58.52	João
2	Calabresa	50.05	Maria

Análise sobre a utilização dos comandos DML

Comandos DML é um conjunto de comandos SQL para manipulação de dados, são

eles: INSERT, DELETE, UPDATE E SELECT.

- SELECT: Comando de acesso, para acessar dados dentro do banco;
- INSERT: Comando de alteração, para a inserção de dados dentro de uma tabela;
- DELETE: Comando de alteração, para deletar dados dentro de uma tabela;
- UPDATE: Comando de alteração, para atualizar dados dentro de uma tabela;

A Linguagem de Manipulação de Dados (DML) é essencial para fazer acessar, analisar e transformar dados dentro de um banco, sendo fundamental para qualquer profissional que utilize banco de dados dentro de seu trabalho.

Referências

<https://www.datacamp.com/pt/tutorial/sql-dml-commands-mastering-data-manipulation-in-sql>

Título da aula: Manipulação de Dados com Comandos DML (Parte II)

Roteiro 4

Scripts SQL utilizados durante a atividade

```
--1. Inserir três registros na tabela Pizzas com preços diferentes.
insert into Pizzas values('4 queijos', 37.8)
insert into Pizzas values('Pepperoni', 52.4)
insert into Pizzas values('Frango', 36.2)
go

/* Após inserir os dados, executar uma consulta utilizando o operador maior que (>)
para listar apenas as pizzas com preço superior a R$ 50,00. */
select * from Pizzas where preco > 50.00
go

--Adicionando a coluna Bairro na tabela Clientes para fazer a proxima query
alter table Clientes
add Bairro varchar(30)
go

--Atualizando os dados dos clientes que estavam sem o Bairro adicionado, adicionando os bairros
update Clientes set Bairro = 'Norte' where id_cliente = 3
update Clientes set Bairro = 'Centro-Oeste' where id_cliente = 4
go

/* adicionado um residente no Bairro Centro e outro residente no Bairro Su l*/
insert into Clientes values('Davih', 9822451, '98797-576', 3, 'Centro')
insert into Clientes values('Isaac', 9554432, '85867-234', 90, 'Sul')
go

/*
- localizar um cliente que mora no Bairro Centro E cujo ID seja igual a 1;
- listar todos os clientes que moram no Bairro Sul OU no Bairro Norte.
*/
select * from Clientes where Bairro = 'Centro' and id_cliente = 1
select * from Clientes where Bairro = 'Sul' or Bairro = 'Norte'
go
```

```
/* registrar um pedido vinculando um cliente a uma pizza.
```

Após o cadastro do pedido, deverão realizar uma consulta utilizando o operador diferente de (<>) para listar todos os pedidos cujo status não seja "Finalizado".*/

```
insert into Pedidos values(5, 15, 'Fazendo')
```

```
insert into Pedidos values(6, 9, 'Finalizado')
```

```
go
```

```
select p.Nome, c.nome, pp.status
```

```
from Pedidos pp
```

```
inner join Pizzas p
```

```
on p.id_pizza = pp.fk_id_pizza
```

```
inner join Clientes c
```

```
on c.id_cliente = pp.fk_id_cliente
```

```
where status <> 'Finalizado'
```

```
go
```

Evidências dos resultado com capturas de tela demonstrando a execução das consultas e manipulações de dados:

Pizzas cadastradas na tabela de pizzas:

	id_pizza	nome	preco
1	8	Margherita	58.52
2	9	Calabresa	50.05
3	10	Portugu...	53.57
4	11	Superpr...	47.31
5	12	4 queijos	37.80
6	15	Pepperoni	52.40
7	16	Frango	36.20

Resultado das pizzas com o preço maior que 50.00:

	id_pizza	nome	preco
1	8	Margherita	58.52
2	9	Calabresa	50.05
3	10	Portuguesa	53.57
4	15	Pepperoni	52.40

Resultado de uma query de filtragem pelo bairro e id do cliente:

```

/*
localizar um cliente que mora no Bairro Centro E cujo ID seja igual a 1;
listar todos os clientes que moram no Bairro Sul OU no Bairro Norte.
*/
select * from Clientes where Bairro = 'Centro' and id_cliente = 1
select * from Clientes where Bairro = 'Sul' or Bairro = 'Norte'
go

```

100 %

Resultados Mensagens

	id_cliente	nome	telefone	cep	casa_numero	Bairro
1	3	João	975662243	6453-452	45	Norte
2	6	Isaac	9554432	85867-234	90	Sul

Resultado da filtragem dos pedidos que estejam com o valor de status diferente do 'finalizado'.

```

select p.Nome, c.nome, pp.status
from Pedidos pp
inner join Pizzas p
on p.id_pizza = pp.fk_id_pizza
inner join Clientes c
on c.id_cliente = pp.fk_id_cliente
where status <> 'Finalizado'
go

```

100 %



Resultados



Mensagens

	Nome	nome	status
1	Margherita	João	Fazendo
2	Calabresa	Maria	Fazendo
3	Pepperoni	Davilh	Fazendo

Utilização dos operadores SQL nas consultas realizadas

Com os operadores SQL conseguimos realizar manipulações com os dados presentes dentro do SQL Server, se tornando uma ferramenta essencial quando se trabalha com banco de dados, nos permitindo fazer filtragem, atualizações, adições ou remoções nas tabelas criadas durante os exercícios do roteiro.

Título da aula: Operadores Especiais em SQL

Roteiro 5

Scripts SQL utilizados durante a atividade

```
/*
Inserir cinco novos registros na tabela Pizzas, com preços variando
entre R$ 35,00 e R$ 85,00.
*/
insert into Pizzas values('Mussarela', 38.7)
insert into Pizzas values('Napolitana', 80.2)
insert into Pizzas values('Lombo canadense', 77.9)
insert into Pizzas values('Romeu e Julieta ', 67.7)
insert into Pizzas values('carne seca', 45.6)
go
--Utilizar o operador LIKE para localizar pizzas cujo nome contenha a palavra "Frango".
select * from Pizzas where nome like '%Frango%'
go
--Utilizar o operador BETWEEN para selecionar pizzas com preço entre R$ 40,00 e R$
60,00.
select * from Pizzas where preco between 40.00 and 60.00
go
/*
Utilizar o operador IN para localizar clientes que residem nos bairros:
- Centro
- Sudoeste
- Jardins
*/
select * from Clientes where Bairro in('Centro', 'Sudoeste', 'Jardins')
go
--Verificação de dados nulos
select * from Pedidos where status is null
go
-- Verificação de existência
select * from Clientes
```

```

where exists(select fk_id_cliente from Pedidos)
go
--Atualização com filtro
update Pizzas set preco = (preco * 0.10) + preco where nome in('4 queijos', 'carne seca',
'Frango')
go
--Remoção de registros de teste
delete Clientes where nome like '%teste%'
go

```

Evidências dos resultados obtidos com capturas de tela demonstrando a execução das consultas

Resultados dos select feitos no script:

100 % <

Resultados Mensagens

	id_pizza	nome	preco
1	16	Frango	39.82

	id_pizza	nome	preco
1	8	Margherita	58.52
2	9	Calabresa	50.05
3	10	Portugu...	53.57
4	11	Superpr...	47.31
5	12	4 queijos	41.58
6	15	Pepperoni	52.40
7	21	came se...	50.16

	id_cliente	nome	telefone	cep	casa_numero	Bairro
1	5	Daviah	9822451	98797-576	3	Centro

	fk_id_cliente	fk_id_pizza	status
1	3	19	NULL
2	5	21	NULL

	id_cliente	nome	telefone	cep	casa_numero	Bairro
1	3	João	975662243	6453-452	45	Norte
2	4	Maria	989804753	4323-744	7	Centro-Oeste
3	5	Da...	9822451	98797-...	3	Centro
4	6	Isaac	9554432	85867-...	90	Sul

Utilização dos operadores especiais em SQL

Com a utilização dos operadores especiais do SQL conseguimos fazer as queries com filtros mais complexos ou específicos, como por exemplo, pegar dados que estão entre uma margem utilizando o operador 'between', dados que são nulos com o 'is null', pegar dados que enquadram em valores que foram especificados com o 'in', verificar se um valor existe com o 'exist' e com o 'like' podemos pegar valores de texto que tenham palavras iguais as quais foram especificadas .

Título da aula: Criação e Utilização de Views em SQL

Roteiro 6

Scripts SQL utilizados para criação das views

```
create view vw_cardapio_economico  
as select nome, preco  
from Pizzas  
where preco > 50.00  
go
```

```
SELECT * FROM vw_cardapio_economico  
go
```

```
create view vw_clientes_contato  
as select nome, telefone  
from Clientes  
go
```

```
SELECT * FROM vw_clientes_contato  
go
```

```
create view vw_calculo as  
select nome, preco as 'preco original', ((preco * 0.10) + preco) as 'preco com taxa'  
from Pizzas  
go
```

```
SELECT * FROM vw_calculo  
go
```

Evidências dos resultados obtidos por meio das views com capturas de tela demonstrando a execução das consultas:

100 % <

Resultados Mensagens

	nome	preco
1	Margherita	58.52
2	Calabresa	50.05
3	Portugue...	53.57
4	Pepperoni	52.40
5	Napolitana	80.20
6	Lombo c...	77.90
7	Romeu e...	67.70
8	came se...	50.16

	nome	telefone
1	João	975662243
2	Maria	989804753
3	Davilh	9822451
4	Isaac	9554432

	nome	preco original	preco com taxa
1	Margherita	58.52	64.3720
2	Calabresa	50.05	55.0550
3	Portugue...	53.57	58.9270
4	Superpro...	47.31	52.0410
5	4 queijos	41.58	45.7380
6	Pepperoni	52.40	57.6400
7	Frango	39.82	43.8020
8	Mussarela	38.70	42.5700
9	Napolitana	80.20	88.2200
10	Lombo c...	77.90	85.6900
11	Romeu e...	67.70	74.4700
12	came se...	50.16	55.1760

Breve explicação sobre as vantagens do uso de views em bancos de dados

As views podem trazer diversas vantagens, como o reuso, invés de escrever o mesmo código várias vezes, podemos criar uma view, e quando precisarmos bastas executá-las, trazem também segurança, já que com a view podemos ocultar algumas colunas da tabela e também simplificam o código, nos permitindo criar linhas de códigos mais limpas.

Referências

<https://www.devmedia.com.br/conceitos-e-criacao-de-views-no-sql-server/22390>

Título da aula: Stored Procedures em SQL

Roteiro 7

Scripts SQL utilizados para criação das procedures

```
create procedure sp_registrar_pedido
    @id_cliente int,
    @id_pizza int
as
begin
    declare @date_now datetime
    set @date_now = getdate()
    insert into Pedidos values(@id_cliente, @id_pizza, 'Em preparação')
    print 'Dado inserido com sucesso, data: ' + convert(varchar, @date_now, 103)
end

exec sp_registrar_pedido 4, 21
exec sp_registrar_pedido 5, 21
exec sp_registrar_pedido 5, 12
go

create procedure sp_fechar_conta
    @id_cliente int,
    @id_pizza int
as
begin
    declare @date_now datetime
    set @date_now = getdate()
    update Pedidos set status = 'Finalizado'
    where fk_id_cliente = @id_cliente and fk_id_pizza = @id_pizza
    print 'Dado atualizado com sucesso, data: ' + convert(varchar, @date_now, 103)
end

exec sp_fechar_conta 5, 15
go
```


Capturas de tela demonstrando a execução das procedures

(1 linha(s) afetadas)

Dado inserido com sucesso, data: 18/04/2026

(1 linha(s) afetadas)

Dado inserido com sucesso, data: 18/04/2026

(1 linha(s) afetadas)

Dado inserido com sucesso, data: 18/04/2026

(1 linha(s) afetadas)

Dado atualizado com sucesso, data: 18/04/2026

Evidências das alterações realizadas nas tabelas

Tabela pedidos antes de chamar as procedures criadas:

	fk_id_cliente	fk_id_pizza	status
1	3	8	Fazendo
2	3	10	Cancelado
3	4	9	Fazendo
4	5	15	Fazendo
5	6	9	Finalizado

Tabela pedidos depois de chamar as procedures criadas:

	fk_id_cliente	fk_id_pizza	status
1	3	8	Fazendo
2	3	10	Cancelado
3	4	9	Fazendo
4	4	21	Em preparação
5	5	12	Em preparação
6	5	15	Finalizado
7	5	21	Em preparação
8	6	9	Finalizado

Utilidade das procedures no contexto de sistemas de informação.

Procedures são essenciais para os sistemas de informação, pois com elas podemos reutilizar código, assim ao invés de ter que reescrever o mesmo código toda vez que for realizar a mesma operação, criamos uma procedure e sempre que precisarmos, basta executá-la. Com elas podemos agilizar o processo de desenvolvimento, sendo uma ferramenta fundamental para quem trabalha com banco de dados.

Título da aula: Controle de Acesso com Comandos DCL

Relatório 8

Scripts SQL para criação de usuários e uso do GRANT e REVOKE

```
--Criação de usuários
CREATE USER atendente_pizzaria FOR LOGIN atendente_pizzaria
WITH DEFAULT_SCHEMA = dbo;
GO

CREATE USER gerente_vendas FOR LOGIN gerente_vendas
WITH DEFAULT_SCHEMA = dbo;
GO

--permissões do gerente_vendas atendente_pizzaria
DENY DELETE ON OBJECT::dbo.Clientes TO atendente_pizzaria;
DENY DELETE ON OBJECT::dbo.Pedidos TO atendente_pizzaria;
DENY DELETE ON OBJECT::dbo.Pizzas TO atendente_pizzaria;

GRANT SELECT ON OBJECT::dbo.Clientes TO atendente_pizzaria;
GRANT SELECT ON OBJECT::dbo.Pizzas TO atendente_pizzaria;
GRANT INSERT ON OBJECT::dbo.Pedidos TO atendente_pizzaria;
GO

--permissões do gerente_vendas
grant select on OBJECT::dbo.Clientes to gerente_vendas
with grant option
go

grant select on OBJECT::dbo.Pizzas to gerente_vendas
with grant option
go

grant select on OBJECT::dbo.Pedidos to gerente_vendas
with grant option
go

grant insert on OBJECT::dbo.Clientes to gerente_vendas
with grant option
go

grant insert on OBJECT::dbo.Pizzas to gerente_vendas
with grant option
go

grant insert on OBJECT::dbo.Pedidos to gerente_vendas
with grant option
go

grant update on OBJECT::dbo.Clientes to gerente_vendas
with grant option
go
```

```
grant update on OBJECT::dbo.Pizzas to gerente_vendas  
with grant option  
go
```

```
grant update on OBJECT::dbo.Pedidos to gerente_vendas  
with grant option  
go
```

```
grant delete on OBJECT::dbo.Clientes to gerente_vendas  
with grant option  
go
```

```
grant delete on OBJECT::dbo.Pizzas to gerente_vendas  
with grant option  
go
```

```
grant delete on OBJECT::dbo.Pedidos to gerente_vendas  
with grant option  
go
```

```
-- Verificando as permissões  
EXEC relatorio_exerc.dbo.sp_helprotect  
    @username = 'atendente_pizzaria'  
GO  
EXEC relatorio_exerc.dbo.sp_helprotect  
    @username = 'gerente_vendas'  
GO
```

```
--Revogar as permissões do atendente_pizzaria  
REVOKE SELECT, INSERT, DELETE, UPDATE ON OBJECT::dbo.Pizzas FROM  
atendente_pizzaria CASCADE;  
REVOKE SELECT, INSERT, DELETE, UPDATE ON OBJECT::dbo.Clientes FROM  
atendente_pizzaria CASCADE;  
REVOKE SELECT, INSERT, DELETE, UPDATE ON OBJECT::dbo.Pedidos FROM  
atendente_pizzaria CASCADE;  
go
```

Capturas de tela demonstrando a execução das operações:

Permissões concedidas para cada um dos usuários:

	Owner	Object	Grantee	Grantor	Protect Type	Action	Column
1	dbo	Cientes	atendente_pizzaria	dbo	Deny	Delete	.
2	dbo	Cientes	atendente_pizzaria	dbo	Grant	Select	(All+N...
3	dbo	Pedidos	atendente_pizzaria	dbo	Deny	Delete	.
4	dbo	Pedidos	atendente_pizzaria	dbo	Grant	Insert	.
5	dbo	Pizzas	atendente_pizzaria	dbo	Deny	Delete	.
6	dbo	Pizzas	atendente_pizzaria	dbo	Grant	Select	(All+N...
7	.	.	atendente_pizzaria	dbo	Grant	CON...	.

	Owner	Object	Grantee	Grantor	Protect Type	Action	Column
1	dbo	Cientes	gerente_vendas	dbo	Grant_WGO	Delete	.
2	dbo	Cientes	gerente_vendas	dbo	Grant_WGO	Insert	.
3	dbo	Cientes	gerente_vendas	dbo	Grant_WGO	Select	(All+N...
4	dbo	Cientes	gerente_vendas	dbo	Grant_WGO	Upd...	(All+N...
5	dbo	Pedidos	gerente_vendas	dbo	Grant_WGO	Delete	.
6	dbo	Pedidos	gerente_vendas	dbo	Grant_WGO	Insert	.
7	dbo	Pedidos	gerente_vendas	dbo	Grant_WGO	Select	(All+N...
8	dbo	Pedidos	gerente_vendas	dbo	Grant_WGO	Upd...	(All+N...
9	dbo	Pizzas	gerente_vendas	dbo	Grant_WGO	Delete	.
10	dbo	Pizzas	gerente_vendas	dbo	Grant_WGO	Insert	.
11	dbo	Pizzas	gerente_vendas	dbo	Grant_WGO	Select	(All+N...
12	dbo	Pizzas	gerente_vendas	dbo	Grant_WGO	Upd...	(All+N...
13	.	.	gerente_vendas	dbo	Grant	CON...	.

Erro caso o usuário tente realizar uma operação não permitida:

```
EXECUTE AS USER = 'atendente_pizzaria';
SELECT USER_NAME() AS UserName;

select * from Pedidos

delete from Pedidos where fk_id_cliente = 6
```

Mensagens

Mensagem 229, Nível 14, Estado 5, Linha 652
A permissão SELECT foi negada no objeto 'Pedidos', banco de dados 'relatorio_exerc', esquema 'dbo'.
Mensagem 229, Nível 14, Estado 5, Linha 652
A permissão DELETE foi negada no objeto 'Pedidos', banco de dados 'relatorio_exerc', esquema 'dbo'.

Análise sobre a importância da segurança em bancos de dados:

Segurança dentro de um banco de dados é algo essencial para o funcionamento de um. Com o uso dos comandos GRANT e REVOKE, podemos criar uma camada de segurança para o nosso banco, com eles podemos permitir ou negar comandos para os usuários. Com isso evitamos acidentes, como por exemplo, um usuário não permitido conseguindo deletar dados de uma tabela, comprometendo a confiabilidade do banco de dados.